GloVe: Global Vectors for Word Representation

Architecture & internals of the Stanford reference implementation

---

A guided tour of `stanfordnlp/GloVe`

October 13, 2026

Roadmap

1. Why word embeddings, why GloVe (Global Vectors)
2. The model: ratios $\rightarrow$ loss
3. The four-stage pipeline
4. vocab_count
5. cooccur
6. shuffle
7. glove: the trainer
8. Operating it: knobs, sizing, evaluation
9. Quirks & summary

Every slide maps to a function in `src/`. The detailed write-up is in `glove_architecture.pdf`.

Abbreviations used in this deck

AdaGrad	Adaptive gradient (optimiser)	OOV	Out-of-vocabulary
CREC	Co-occurrence record (16-byte struct)	PPMI	Positive pointwise mutual information
GB	Gigabyte	pthreads	POSIX threads
GloVe	Global Vectors for Word Representation	RAM	Random-access memory
I/O	Input/output	SGD	Stochastic gradient descent
Inf	Infinity (floating point)	SSD	Solid-state drive
LSA	Latent Semantic Analysis	stdin	Standard input
NaN	Not a number (floating point)	stdout	Standard output
nnz	Number of non-zero entries (of X)	stderr	Standard error
		SVD	Singular value decomposition

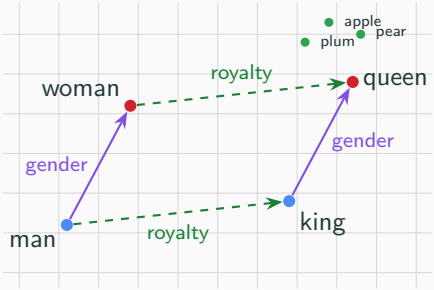
Venue abbreviations (EMNLP, NeurIPS, JMLR) are spelled out in the References.

Motivation

What is a word embedding?

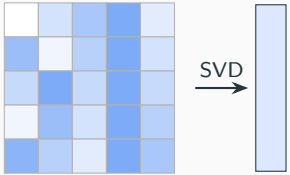
- Map each word i to a dense vector $\mathbf{w}_i \in \mathbb{R}^d$ ($d \approx 50$–300).
- Similar words end up close together.
- Relations become directions:

$$\mathbf{w}_{\text{king}} - \mathbf{w}_{\text{man}} + \mathbf{w}_{\text{woman}} \approx \mathbf{w}_{\text{queen}}$$



Two families before GloVe — and the bridge

Count-based (LSA, PPMI + SVD)

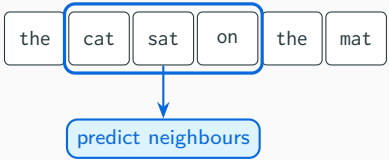


global counts X

+ uses all statistics

– poor analogies

Predictive (word2vec)



+ linear structure

– revisits every window

GloVe: count *once* globally, then fit a log-bilinear model by stochastic gradient descent (SGD)

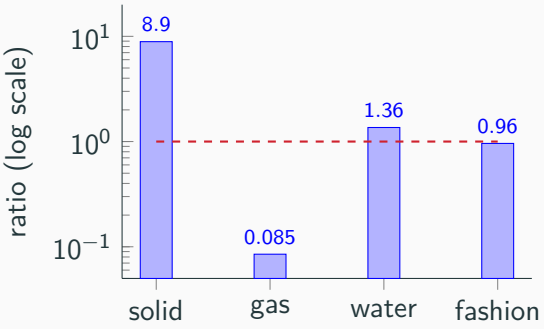
The Model

Meaning lives in probability *ratios*

probe k	$\frac{P(k\mid\text{ice})}{P(k\mid\text{steam})}$	reading
solid	**8.9**	ice-specific
gas	**0.085**	steam-specific
water	1.36	both
fashion	0.96	neither

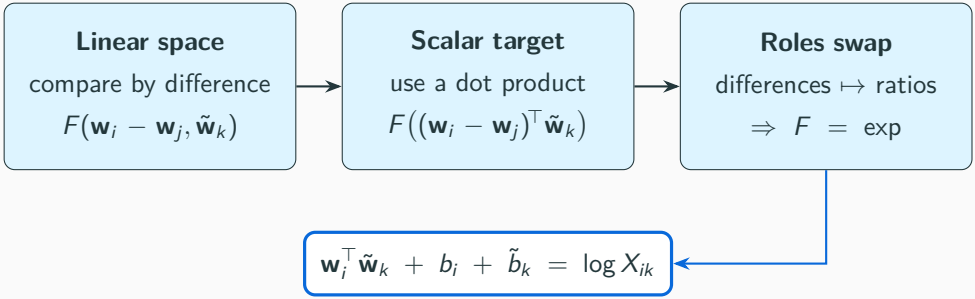
Raw probabilities (10^{-5}–10^{-3}) are hard to read. The ratio separates relevant from irrelevant probe words.

Source: Pennington et al. 2014, Table 1 (6B-token corpus).



dashed line: ratio = 1 (uninformative)

From ratios to a model in three steps



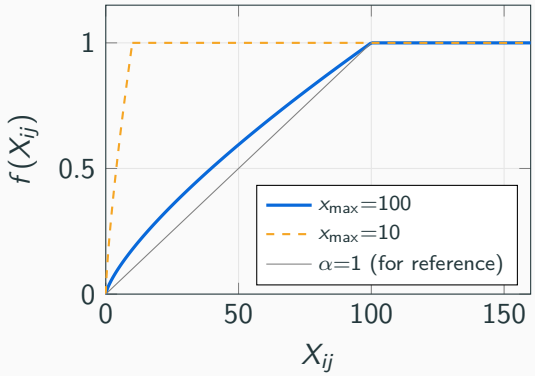
$\exp(\mathbf{w}_i^\top \tilde{\mathbf{w}}_k) = P_{ik} = X_{ik}/X_i \Rightarrow \log X_i$ does not depend on k, so it becomes the bias b_i; $\tilde{b}_k$ is added for symmetry.

The objective: weighted least squares on $\log X$

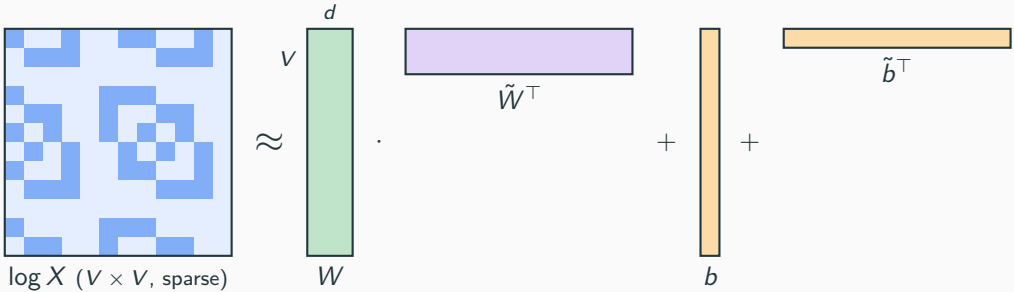
$$J = \sum_{i,j:\, X_{ij}>0} f(X_{ij}) \Big(\underbrace{\mathbf{w}_i^\top \tilde{\mathbf{w}}_j + b_i + \tilde{b}_j - \log X_{ij}}_{\delta_{ij}\ \texttt{(diff)}} \Big)^2$$

$$f(x) = \begin{cases} (x/x_{\max})^\alpha & x < x_{\max} \\ 1 & \text{otherwise} \end{cases}$$

- Sum only over non-zero cells, so the data can be stored sparse.
- $f(0) = 0$ removes the $\log 0$ problem.
- Rare (noisy) pairs get small weight; frequent pairs are capped at weight 1.
- $\alpha = 0.75$, $x_{\max} = 100$ (paper); `demo.sh` uses $x_{\max} = 10$.



GloVe is a weighted factorisation of $\log X$



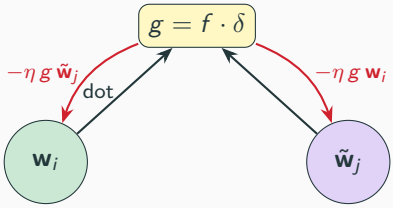
- Only the dark cells (observed pairs) enter the loss, each with weight $f(X_{ij})$.
- Cost per epoch is $O(\mathrm{nnz}(X) \cdot d)$, where nnz is the number of non-zero cells, independent of corpus length.
- There are two tables, W and $\tilde{W}$. The default output is $\mathbf{w}_i + \tilde{\mathbf{w}}_i$, a small ensemble.

Gradients (what the inner loop computes)

With $\delta = \mathbf{w}_i^\top \tilde{\mathbf{w}}_j + b_i + \tilde{b}_j - \log X_{ij}$ and $g = f(X_{ij})\,\delta$ (`fdiff`):

$$\nabla_{\mathbf{w}_i} = g\,\tilde{\mathbf{w}}_j \qquad \nabla_{\tilde{\mathbf{w}}_j} = g\,\mathbf{w}_i$$
$$\partial_{b_i} = g \qquad \partial_{\tilde{b}_j} = g$$

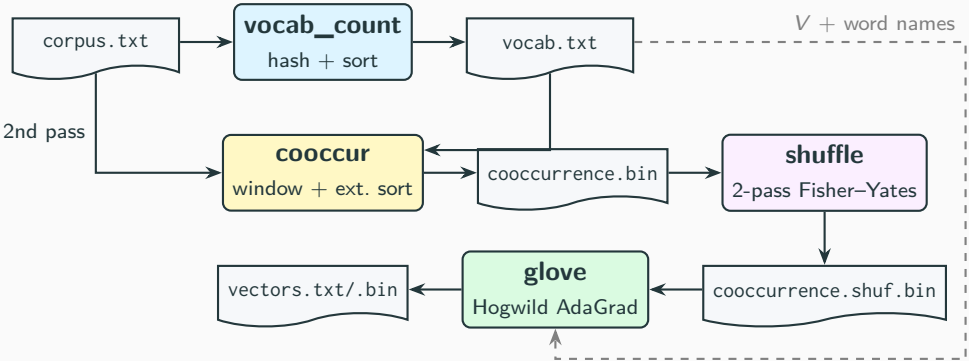
The reported cost is $\frac{1}{2} f \delta^2$, so the factor 2 drops out.



each vector is pulled using the *other* vector's old value

The Pipeline

Four Unix filters, connected by files



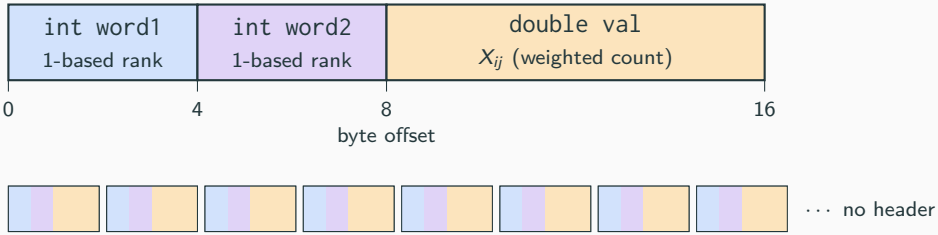
- Each tool reads standard input (`stdin`) or files, writes standard output (`stdout`), and logs to standard error (`stderr`). There is no shared state.
- Any stage can be re-run alone (e.g. retrain with a new d without recounting).
- Every stage except `glove` works out of core.

Running it end to end (demo.sh)

```
make                                          # -> build/{vocab_count,cooccur,shuffle,glove}
build/vocab_count -min-count 5 -verbose 2                    < text8 > vocab.txt
build/cooccur -memory 4.0 -vocab-file vocab.txt -window-size 15 < text8 > cooccurrence.bin
build/shuffle -memory 4.0                          < cooccurrence.bin > cooccurrence.shuf.bin
build/glove -save-file vectors -threads 8 -input-file cooccurrence.shuf.bin \
            -x-max 10 -iter 15 -vector-size 50 -binary 2 -vocab-file vocab.txt
python eval/python/evaluate.py                # analogy accuracy
```

- text8 is about 17M tokens; the demo runs in minutes.
- The 2024 Wiki+Gigaword runs used `-min-count` 20, `-window-size` 10, and 50–100 iterations.
- The corpus must be pre-tokenised and whitespace-separated.
- A newline marks a document boundary: context windows never cross it.

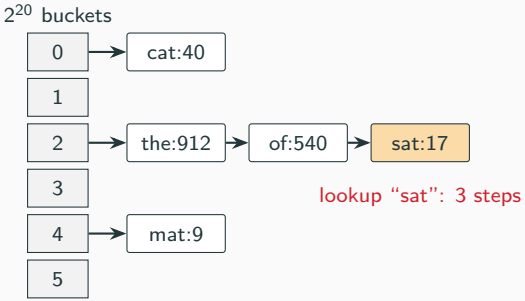
The universal record: CREC (co-occurrence record, 16 bytes)



- Files written by cooccur and shuffle and read by glove are raw fwrites of CREC arrays.
- size $= 16 \cdot \mathrm{nnz}(X)$. glove computes the record count as filesize/16.
- The layout is native-endian and not portable across architectures.

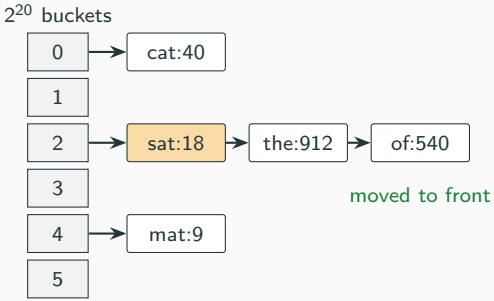
Stage 1: vocab_count

Counting words: hash table with move-to-front



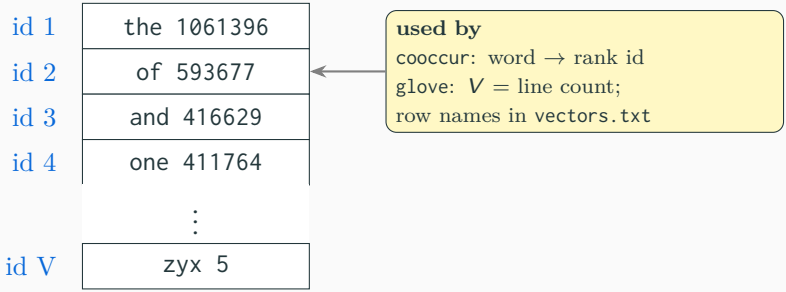
- `bitwisehash`: shift-xor string hash.
- Separate chaining. A hit is moved to the front of its chain.
- Under Zipf's law a few words get most lookups, so they stay at the front and lookups are close to $O(1)$.
- Then the words are sorted by count desc (ties alphabetical) and the list is truncated by `-min-count`/`-max-vocab`.

Counting words: hash table with move-to-front



- `bitwisehash`: shift-xor string hash.
- Separate chaining. A hit is moved to the front of its chain.
- Under Zipf's law a few words get most lookups, so they stay at the front and lookups are close to $O(1)$.
- Then the words are sorted by count desc (ties alphabetical) and the list is truncated by `-min-count`/`-max-vocab`.

The vocab.txt contract: *line number = word id*



- Id 1 is the most frequent word. Small ids are the hot words, which cooccur exploits.
- The literal token <unk> is forbidden in the corpus because it is reserved for the output.
- Never edit vocab.txt between stages.

Stage 2: cooccur

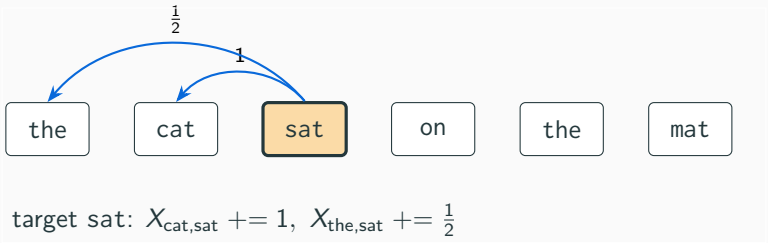
Sliding the window (symmetric, $W=2$, weight $1/\text{distance}$)



target `cat`: $X_{\text{the,cat}} \mathrel{+}= 1$ (and the transpose)

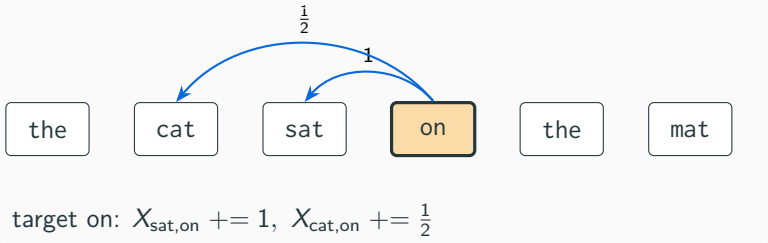
- The tool only looks left, using a circular `history[W]` buffer. "Symmetric" adds X_{ji} as well.
- Out-of-vocabulary tokens are skipped completely: they do not take a slot or count toward distance.
- A newline resets the window.

Sliding the window (symmetric, $W{=}2$, weight $1/\text{distance}$)



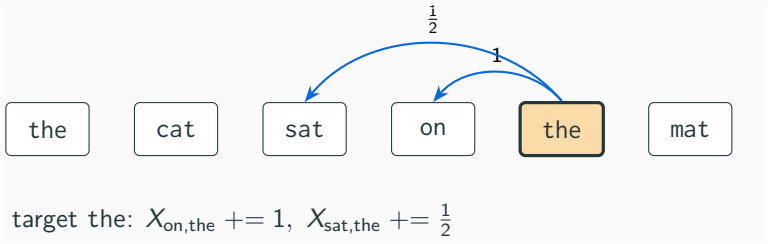
- The tool only looks left, using a circular `history[W]` buffer. "Symmetric" adds X_{ji} as well.
- Out-of-vocabulary tokens are skipped completely: they do not take a slot or count toward distance.
- A newline resets the window.

Sliding the window (symmetric, $W{=}2$, weight $1/\text{distance}$)



- The tool only looks left, using a circular `history[W]` buffer. "Symmetric" adds X_{ji} as well.
- Out-of-vocabulary tokens are skipped completely: they do not take a slot or count toward distance.
- A newline resets the window.

Sliding the window (symmetric, $W{=}2$, weight $1/\text{distance}$)



- The tool only looks left, using a circular `history[W]` buffer. "Symmetric" adds X_{ji} as well.
- Out-of-vocabulary tokens are skipped completely: they do not take a slot or count toward distance.
- A newline resets the window.

Sliding the window (symmetric, $W=2$, weight $1/\text{distance}$)



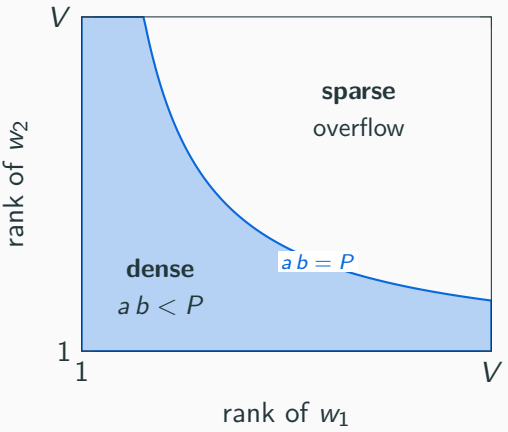
- The tool only looks left, using a circular `history[W]` buffer. "Symmetric" adds X_{ji} as well.
- Out-of-vocabulary tokens are skipped completely: they do not take a slot or count toward distance.
- A newline resets the window.

Result: the co-occurrence matrix of the example

		the	cat	mat	on	sat
1	the		1	1	1	1
2	cat	1			0.5	1
3	mat	1			0.5	
4	on	1	0.5	0.5		1
5	sat	1	1		1	

- Ids: the=1, then ties alphabetical: cat=2, mat=3, on=4, sat=5.
- The matrix is symmetric because the context is symmetric.
- $X_{\text{the,sat}} = \frac{1}{2} + \frac{1}{2}$ comes from two distance-2 hits.
- 16 non-zero cells, so 16 CRECs = 256 bytes.
- The diagonal and empty cells are never stored.

Zipf ⇒ split the matrix by rank product



- Count of the rank-r word $\propto 1/r$, so $\mathbb{E}[X_{ab}]$ falls roughly like $1/(ab)$.
- **Dense region** (small ab): hot and heavily filled. Stored in a ragged array and updated in place.
- **Sparse region**: cold, long tail. Appended to a buffer, then sorted and spilled to disk.
- $P =$ `max_product`, derived from `-memory`.

The dense part: a ragged array + prefix sums

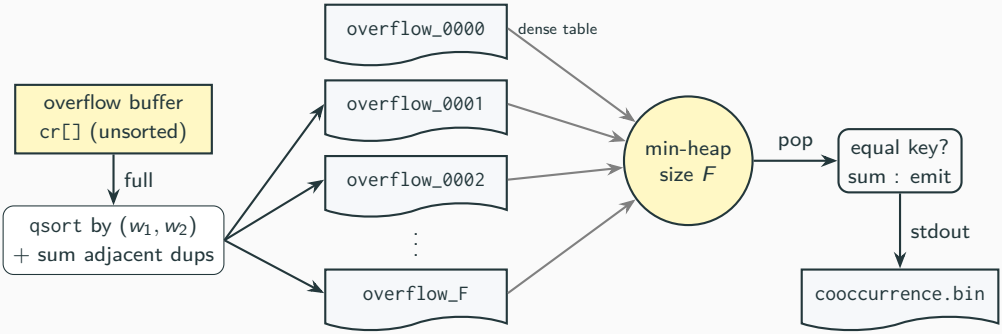
		column w_2					
		1	2	3	4	5	6
lookup[0]=1	row 1	0	1	2	3	4	5
lookup[1]=7	row 2	6	7	8	9	10	11
lookup[2]=13	row 3	12	13	14	15		
lookup[3]=17	row 4	16	17	18			
lookup[4]=20	row 5	19	20				
lookup[5]=22	row 6	21	22				

Toy example: $P = 12$, $V = 6$; cells show the offset in `bigram_table`.

```
lookup[0] = 1;
for (a = 1; a <= V; a++)
  lookup[a] = lookup[a-1]
            + min(P / a, V);
// cell (w1,w2), 1-based:
bigram_table[lookup[w1-1]
             + w2 - 2] += 1.0/dist;
```

Highlighted: $(3, 2) \mapsto 13 + 2 - 2 = 13$.
Row a holds $\min(\lfloor P/a \rfloor, V)$ cells.

The sparse part: spill sorted runs, then a k-way merge



The output is globally sorted with unique (w_1, w_2) keys, and temp files are deleted afterwards. More `-memory` means more pairs take the dense path and fewer files are merged. The *output is identical* either way.

Turning -memory into buffer sizes

$$R = \frac{0.85 \cdot M \cdot 2^{30}}{16}, \qquad \texttt{overflow_length} = R/6$$

Solve for P by fixed-point iteration:

$$P\,(\ln P + \underbrace{0.1544313298}_{2\gamma-1}) = R$$

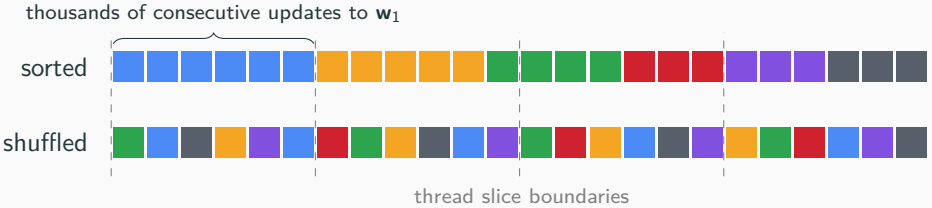
Number of integer pairs with $ab \le P$ $= \sum_{k \le P} d(k) \approx P \ln P + (2\gamma - 1)P$ (Dirichlet divisor estimate). This sizes the hyperbolic dense region.



- Dense cells are 8 B but are divided by 16, so the estimate is conservative.
- Override with `-max-product` and `-overflow-length`.

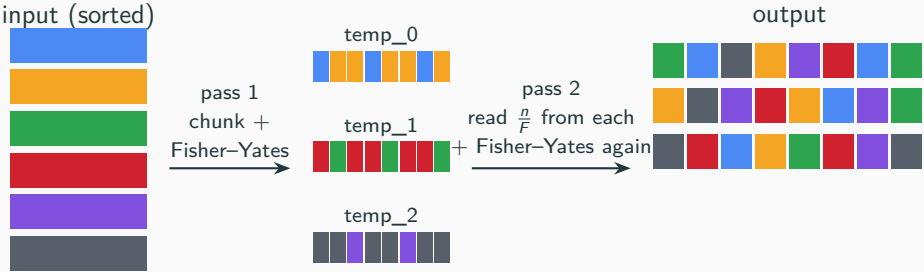
Stage 3: shuffle

Why shuffle? Sorted input breaks SGD



- `cooccurrence.bin` is sorted by w_1, i.e. grouped by word (one colour per word).
- In sorted order, the updates are biased: one row is hammered, then the next.
- Sorted order also causes contention between Hogwild threads. After shuffling, consecutive and concurrent updates touch different rows.

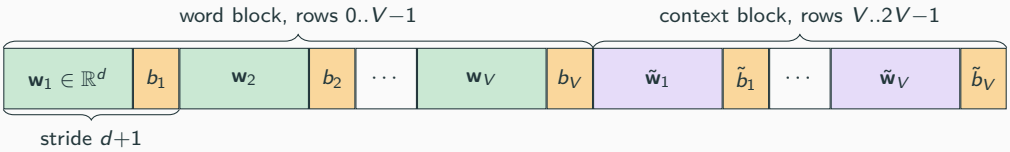
Out-of-core shuffle in two passes



- Chunk size $= 0.95\,M \cdot 2^{30}/16$ records. Fisher–Yates with `rand_long` (rejection sampling, so no modulo bias).
- The result is not a perfectly uniform permutation ("good enough"), but every output window mixes the whole input.
- `-seed` makes it reproducible; the default seed is `time(0)`.

Stage 4: glove, the trainer

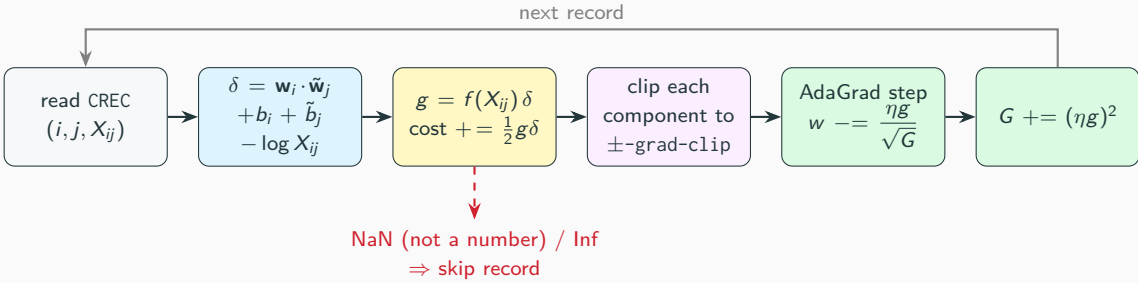
All parameters in one aligned array



record (i, j): $\quad$ `l1`$= (i-1)(d+1)$ $\qquad$ `l2`$= (j-1+V)(d+1)$

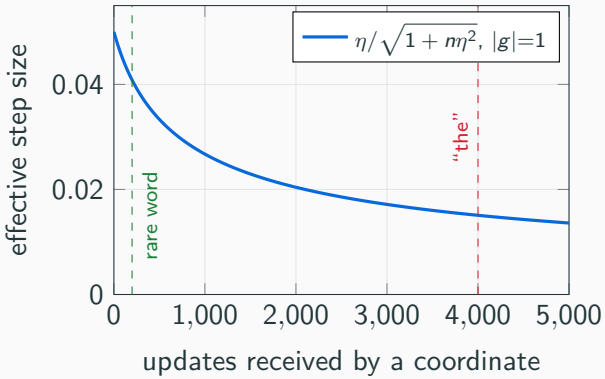
- `W` and `gradsq` each hold $2V(d+1)$ doubles, aligned with `posix_memalign(128)`.
- The bias sits next to its vector, so they share cache lines.
- Initialisation: $W \sim \mathcal{U}(-0.5, 0.5)/d$, `gradsq` $= 1$, or loaded from a checkpoint.

One SGD step, as a pipeline



- Both vector updates are computed from *old* values, then applied together.
- G starts at 1, so the first step size is exactly $\eta\, g$.
- Biases: $b \mathrel{-}= g/\sqrt{G}$, with no η and no clipping.
- The divisor uses G *before* this step's gradient is added.

AdaGrad: frequent words slow down, rare words keep learning



Illustrative: constant $|g| = 1$, $\eta = 0.05$.

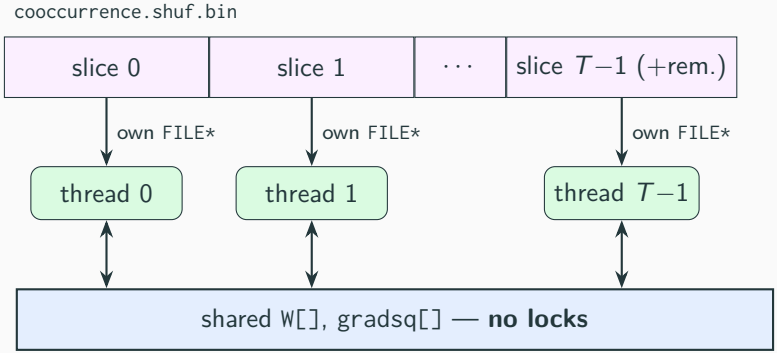
- Each coordinate keeps its own accumulator G_k.
- Frequent words appear in many records, so G grows quickly and their steps shrink.
- Rare words keep taking large steps on their few observations.
- This suits Zipfian data, and there is no learning-rate schedule to tune.

The inner loop in code (glove_thread)

```
l1 = (cr.word1 - 1) * (d+1);   l2 = (cr.word2 - 1 + V) * (d+1);
diff  = dot(W+l1, W+l2, d) + W[l1+d] + W[l2+d] - log(cr.val);
fdiff = (cr.val > x_max) ? diff : pow(cr.val / x_max, alpha) * diff;
if (isnan(diff) || isinf(fdiff) ...) continue;
cost[id] += 0.5 * fdiff * diff;
for (b = 0; b < d; b++) {
    t1 = clip(fdiff * W[b+l2]) * eta;   t2 = clip(fdiff * W[b+l1]) * eta;
    u1[b] = t1 / sqrt(gradsq[b+l1]);    gradsq[b+l1] += t1*t1;
    u2[b] = t2 / sqrt(gradsq[b+l2]);    gradsq[b+l2] += t2*t2;
}
if (finite(sum(u1)) && finite(sum(u2))) for b: { W[b+l1] -= u1[b]; W[b+l2] -= u2[b]; }
W[l1+d] -= fdiff / sqrt(gradsq[l1+d]);   W[l2+d] -= fdiff / sqrt(gradsq[l2+d]);   // biases
gradsq[l1+d] += fdiff*fdiff;             gradsq[l2+d] += fdiff*fdiff;
```

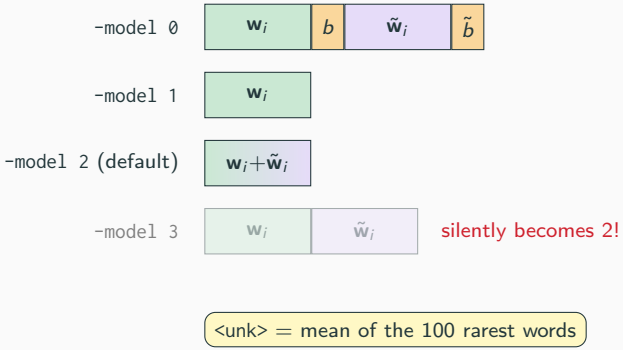
Condensed from src/glove.c, glove_thread().

Hogwild: lock-free parallel SGD



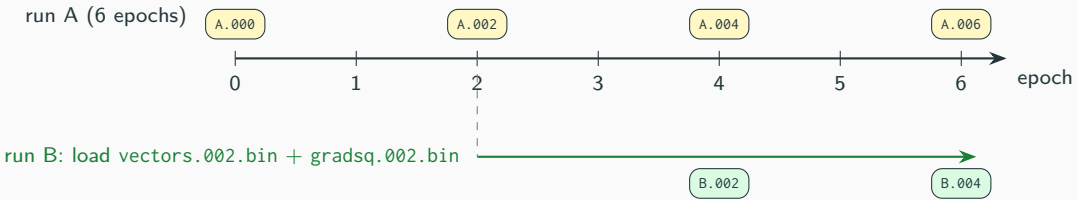
- Each epoch: spawn T POSIX threads (pthreads), `fseeko` each one to its slice, then join.
- Each update touches only $2(d+1)$ of the $2V(d+1)$ entries.
- Collisions are rare and only cost a stale or lost update, which SGD tolerates.
- Multithreaded runs are not bit-reproducible, even with `-seed`.

What gets written



- `-binary`: 0 = text, 1 = raw `W` dump ($2V \times (d+1)$ doubles), 2 = both.
- `-write-header 1` writes "V d" for gensim (the count excludes <unk>).
- The <unk> row is a proxy for unseen words, which is why <unk> is banned from the corpus.

Checkpoints and exact resume



B.002 ≡ A.004 B.004 ≡ A.006 (byte-identical, 1 thread)

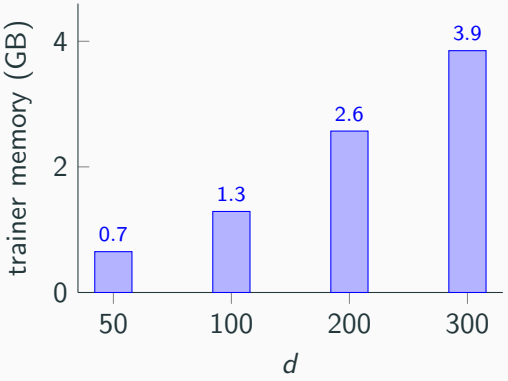
- `-checkpoint-every` k, `-save-init-param` (writes epoch 000), `-gradsq-file`.
- Resume with `-load-init-param 1` and `-load-init-gradsq 1`. This is verified by `randomization.test.sh`.

Operating GloVe

The knobs that matter

tool	flag	default	guidance
vocab_count	`-min-count`	1	5 for text8; 20 for Wiki+Gigaword
	`-max-vocab`	none	1.2M for the Dolma run
cooccur	`-window-size`	15	small: syntax; large: topic (10 in 2024 runs)
	`-symmetric`, `-distance-weighting`	1, 1	left-only / unweighted variants
	`-memory`	3 GB	help says 4; affects speed, not the result
shuffle	`-memory`, `-seed`	2 GB, time	set `-seed` for reproducibility
glove	`-vector-size`	50	50–300
	`-iter`	25	50–100 for large corpora
	`-eta`	0.05	applies to vectors only, not biases
	`-x-max`, `-alpha`	100, 0.75	$x_{\max} \approx 10$ for small corpora
	`-threads`	8	use 1 for bit-reproducibility

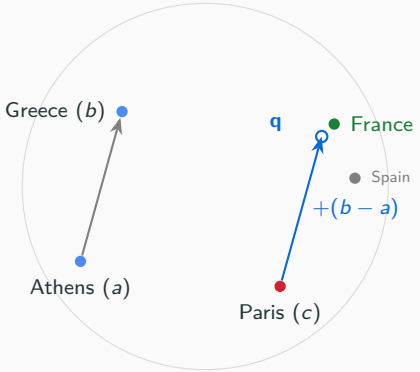
Sizing a run



$V = 400\text{k}$: $2 \cdot 2V(d{+}1) \cdot 8$ bytes (`W` + `gradsq`)

- **Disk**: $16 \cdot \mathrm{nnz}(X)$ for each co-occurrence file. While shuffling there are input, temp, and output copies, so plan for about $3\times$.
- **Time per epoch**: $O(\mathrm{nnz}(X) \cdot d)$. The paper finds that nnz grows sub-linearly in corpus size (about $|C|^{0.8}$).
- **I/O (input/output)**: every stage streams sequentially. `glove` re-reads the whole file every epoch, so a solid-state drive (SSD) or the page cache helps.

Evaluation: word analogies



1. L2-normalise every vector.
2. $\mathbf{q} = \hat{\mathbf{w}}_b - \hat{\mathbf{w}}_a + \hat{\mathbf{w}}_c$.
3. Predict $\arg\max_x \hat{\mathbf{w}}_x^\top \mathbf{q}$, excluding a, b, c.
4. Report top-1 accuracy, split into semantic (5 files) and syntactic (9 files).

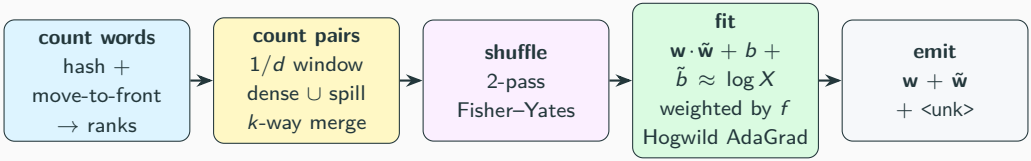
`eval/python/evaluate.py` batches 100 questions per matrix product. `distance.py` and `word_analogy.py` are interactive tools.

Quirks found while reading the code

- **`-model 3` is unreachable**: `glove.c:523` maps everything except 0 and 1 to 2.
- **Bias updates ignore `-eta`** and are not clipped.
- **Defaults vs. help**: `cooccur` defaults to 3 GB and `shuffle` to 2 GB; both help texts say 4.
- **NaN guard vs. gradsq**: a rejected vector update has already incremented `gradsq`.
- **`-Ofast`** disables the NaN checks (`-ffinite-math-only`); the Makefile uses `-O3`.
- **Help invocation**: `cooccur`/`glove` print help with no arguments; `vocab_count`/`shuffle` need `-h`.
- **Out-of-vocabulary (OOV) tokens are skipped**, so window distances count only in-vocabulary tokens.

Summary

GloVe in one picture



Statistics are collected once and globally. Training costs $O(\mathrm{nnz}(X)\cdot d)$ per epoch.
Ratios of co-occurrence probabilities become vector differences.

References

1. J. Pennington, R. Socher, C. D. Manning. *GloVe: Global Vectors for Word Representation*. Empirical Methods in Natural Language Processing (EMNLP) 2014.
2. T. Mikolov et al. *Distributed Representations of Words and Phrases and their Compositionality*. Neural Information Processing Systems (NeurIPS) 2013.
3. J. Duchi, E. Hazan, Y. Singer. *Adaptive Subgradient Methods* (AdaGrad). Journal of Machine Learning Research (JMLR) 2011.
4. B. Recht, C. Ré, S. Wright, F. Niu. *Hogwild!* NeurIPS 2011.
5. `https://nlp.stanford.edu/projects/glove/` — pre-trained vectors and the paper.
6. Companion document: `docs/architecture/glove_architecture.pdf`.